OBA Contracts Audit Report

Updated: 10th June, 2026

Table of Contents

Introduction

The purpose of this report is to document the security analysis and contract structure of the OBA (Offshore Builders Account) fixed deposit contract suite. OBA is a business-banking dApp where users lock USDT or gPRLZ into fixed-term deposits and earn yield based on their EVT (Ecosystem Value Token) tier, which is determined by ownership of Atlantus tokenized land NFTs — ACRE, PLOT, and YARD. This audit examines deposit lifecycle management, yield accounting, payout flows, APY resolution, and the genesis key deposit reward integration for secure and controlled operation of the platform.

Disclaimer

This report is based on the information provided at the time of the audit and does not guarantee the absence of future vulnerabilities. Subsequent security reviews and on-chain monitoring are strongly recommended.

Scope of Audit

The audit focused on the following aspects of the OBA contract suite:

- Security mechanisms, including access controls, role separation, and EIP-712 signature validation across deposit, payout, and manager flows
- Code correctness, fund accounting integrity, and logical flow across all lifecycle stages (deposit, top-up, yield claim, withdrawal, compound)
- Adherence to best practices for upgradeability, EIP-7201 namespaced storage, and storage gap management
- Integration correctness between `FixedDeposits`, `FixedDepositsPayout`, `FixedDepositsManager`, `FixedDepositsGetters`, and `AirdropGenKey`

- Gas efficiency, error handling, and prevention of replay attacks or unauthorized state mutations

---

Methodology

The audit process involved:

- Manual code review of inheritance chains, role enforcement, cross-contract call flows, and fund routing logic
- Automated analysis using Slither and Aderyn to detect common vulnerabilities such as reentrancy, integer overflow, and access control issues
- Scenario-based testing using Foundry for simulating deposit creation, yield accrual, principal withdrawal, compound operations, APY tier resolution, and deposit reward claiming edge cases

---

Security Review Summary

Review commit hash:

- `68412f043a4cc5b0621644174f5965303f71cf10`

Contracts in Scope:

- `FixedDeposits.sol`
- `FixedDepositsPayout.sol`
- `FixedDepositsGetters.sol`
- `FixedDepositsManager.sol`

All Initial issues were fixed and the following number of issues were found, categorised by their severity:

- **High: 0 issues**
- **Medium: 0 issues**
- **Low: 0 issues**

Security Analysis

FixedDeposits

`FixedDeposits` is the core accounting kernel of the OBA platform. It inherits from OpenZeppelin's `Initializable`, `AccessControlUpgradeable`, and `EIP712Upgradeable`, giving it safe initialization, role-gated administration, and typed signature support. All critical state is isolated in a namespaced EIP-7201 storage struct (`FixedDepositsStorage`) to prevent proxy storage collisions across upgrades.

Access control is enforced across four distinct roles: `DEFAULT_ADMIN_ROLE` governs payout and manager contract registration; `OPERATOR_ROLE` handles configuration (APY tiers, custodian, penalty tiers, asset address); `PAYOUT_ROLE` gates all state-mutating payout operations; and `MANAGER_ROLE` gates whitelist and on-behalf deposit management. This layered separation ensures no single role can unilaterally drain funds or alter yield parameters.

Deposit creation via `deposit()` enforces whitelisting and COA verification before accepting any funds. Payment resolution through `_resolvePaymentToUsdt` handles USDT, gPRLZ, and dual-payment modes correctly, routing gPRLZ through the cashier contract's `withdrawOnBehalf` mechanism for accurate USDT-equivalent pricing. The fund settlement logic in `_settleDepositFunds` correctly apportions referral rewards, cashier fees, agent fees, and custodian transfers — with all amounts balanced against the received principal.

Yield calculations use quarter-based fixed-point arithmetic scaled to the cycle duration, preventing truncation errors for small principals or short elapsed times. The `_updateYield` and `_checkpointDepositApy` functions correctly handle mid-cycle APY changes by snapshotting accumulated yield before applying the new rate, ensuring no yield is lost or double-counted during EVT tier upgrades.

Automated tools (Slither/Aderyn) flagged no high-severity issues. Testing confirmed correct behaviour across early withdrawal, penalty application, and matured deposit flows.

FixedDepositsPayout

`FixedDepositsPayout` owns all token movement and request lifecycle logic for the OBA platform. It inherits `AsyncRequest` for a two-phase request pattern — requests are created on user action, fulfilled by a backend-signed `processWithdrawRequests`, and completed by the user — which ensures no funds move until the backend has validated the request off-chain.

All fund-moving operations (`completePrincipalWithdrawRequests`, `completeYieldWithdrawRequests`, `bulkWithdrawYieldClaims`) are guarded by EIP-712 signature verification via `OBASigVerifier`, with per-address nonces preventing replay. The `withdrawToken` and `adminDepositToken` admin functions are similarly signature-gated, meaning even `OPERATOR_ROLE` holders cannot move funds without a valid signer-produced signature.

Yield payouts use a dual-asset model: USDT is routed to the cashier contract and an equivalent amount of gPRLZ is minted to the recipient via `_distributeYieldDual`. This correctly mirrors the OBA economic model without requiring the payout contract to hold gPRLZ balances. The `compoundYield` function handles both pure-yield compounding and hybrid top-up compounding, with payment resolution delegated to the same `_resolvePaymentToUsdt` helper used by `FixedDeposits`, ensuring consistent gPRLZ pricing.

Split distribution via `_distribute` and `_distributeYieldDual` enforces that basis points sum exactly to `BP` (10 000) before completing any transfer, preventing partial or over-distributed payouts.

FixedDepositsGetters

`FixedDepositsGetters` is a stateless, read-only lens contract that performs all rich frontend calculations without adding bytecode weight to `FixedDeposits`. It reads raw deposit fields via `IFixedDepositsGettersSource` and replicates the yield calculation logic of the core contract for view purposes.

The `_tryGetEquivalentEvtCount` function implements a graceful fallback: it first attempts `coaAuth.getNftDetails` for granular NFT type resolution, then falls back to `coaAuth.getTotalKeysStaked` for older COAAuth versions. This makes the getter resilient across COAAuth contract upgrades without requiring a redeployment of the getter itself.

APY resolution in `_getDepositApyBps` correctly falls back to the legacy `userApyById` mapping when a deposit's stored APY is zero — maintaining backward compatibility for deposits created before the snapshot system was introduced. All public view functions are safe for high-frequency off-chain polling with no state side-effects.

FixedDepositsManager

`FixedDepositsManager` externalises the signature-verification and EVT APY resolution workflows that would otherwise increase `FixedDeposits` bytecode size. It interacts with `FixedDeposits` exclusively through the `IFixedDepositsManagerCore` interface and the `MANAGER_ROLE`, maintaining a clean separation of concerns.

Whitelist addition and removal are both gated by EIP-712 signatures verified against the `FixedDeposits` domain separator and nonce storage via `OBASigVerifier`, ensuring the manager cannot whitelist users without a valid backend signature. The `depositOnBehalf` flow additionally validates COA user registration and KYC verification before creating the deposit, matching the security guarantees of the user-facing `deposit()` path.

APY resolution via `resolveDepositApy` applies the same `_tryGetEquivalentEvtCount` → `_getEvtApyBps` pipeline used in `FixedDepositsGetters`, with a local default fallback (`defaultSingleEvtApyBps`, `defaultNineEvtApyBps`, `defaultFiftyFourEvtApyBps`) in case the on-chain EVT config in `FixedDeposits` has not yet been set. The `APY_SYNC_ROLE`-gated `syncUserApy` function allows backend services to push updated APY snapshots to a user's active deposits after an NFT tier change, without requiring user interaction.

Contract Structure

FixedDeposits

- **Storage:** EIP-7201 namespaced `FixedDepositsStorage` struct containing asset, custodian, APY tiers, penalty tiers, cycle config, whitelist states, and deposit records; supplementary top-level mappings for lifetime deposits, user deposit IDs, and early-withdrawal flags
- **Roles:** `DEFAULT_ADMIN_ROLE`, `OPERATOR_ROLE`, `PAYOUT_ROLE`, `MANAGER_ROLE`
- **Initialization:** Sets all core addresses, penalty tiers, cycle configuration, EVT APY defaults, and referral rate parameters
- **Core Functions:**
  - `deposit`: Whitelist + COA check, payment resolution, APY snapshot, referral settlement, airdrop reward recording
  - `increaseDeposit`: Top-up before deposit start, refreshes APY snapshot, updates lifetime deposits and airdrop rewards
  - `consumeWithdraw`: Called by payout contract; applies penalty/tax, removes deposit from active set
  - `consumeYieldClaim` / `consumeClaimableYields`: Called by payout contract; enforces claim frequency and claimable yield ceiling
  - `createCompoundDepositFromPayout`: Called by payout contract; creates new deposit from compounded yield
  - `syncUserApyFromManager` / `createDepositOnBehalfFromManager`: Manager-role entry points for APY sync and admin deposit

- **Internals:** `_calcYield`, `_calcYieldSince`, `_updateYield`, `_checkpointDepositApy`, `_settleDepositFunds`, `_resolvePaymentToUsdt`, `_resolveDepositApy`
- **Getters:** Deposit record, whitelist status, penalty tiers, cycle config, APY config, user deposit IDs

---

FixedDepositsPayout

- **Storage:** EIP-7201 namespaced `PayoutStorage` struct containing deposits contract reference, COAAuth reference, signer, and per-address nonces; inherits `AsyncRequest` for request lifecycle state
- **Roles:** `DEFAULT_ADMIN_ROLE`, `OPERATOR_ROLE`
- **Initialization:** Sets deposits contract, COAAuth, signer, and admin roles; initialises EIP-712 and AsyncRequest
- **Core Functions:**
  - `requestWithdraw`: Calls `consumeWithdraw`, enqueues `PRINCIPAL_WITHDRAWAL` async request
  - `requestClaimYield` / `requestClaimYields`: Calls `consumeYieldClaim` / `consumeClaimableYields`, enqueues `YIELD_CLAIM` requests
  - `compoundYield`: Converts claimable yield + optional top-up into a new deposit via `createCompoundDepositFromPayout`
  - `processWithdrawRequests`: EIP-712 signature-gated request fulfilment
  - `completePrincipalWithdrawRequests`: Completes fulfilled withdrawal requests, distributes USDT
  - `completeYieldWithdrawRequests` / `bulkWithdrawYieldClaims`: Completes yield claims with dual-asset USDT/gPRLZ payout
  - `adminDepositToken` / `withdrawToken`: Signature-gated admin fund management
- **Internals:** `_distribute`, `_distributeYieldDual`, `_resolvePaymentToUsdt`, `_validateRequestTypes`, `_validateFulfilledYieldClaimRequests`
- **Getters:** Request info, contract addresses, signer, user nonce, domain separator

---

FixedDepositsGetters

- **Storage:** Single `depositsContract` address; no EIP-7201 struct required (read-only lens)
- **Initialization:** Sets deposits contract address
- **Core View Functions:**
  - `getDepositInfo`: Returns depositor ID, principal, claimable yield, start date, and effective APY
  - `getClaimableYield`: Computes claimable yield from raw storage fields
  - `getMaturedYieldPerSecond`: Returns the instantaneous yield accrual rate for a deposit
  - `getCurrentApy` / `getCurrentEvtAndApy`: Resolves the caller's current EVT count and applicable APY tier
  - `isDepositMatured` / `isEarlyWithdrawalAllowed`: Deposit state helpers
  - `cycleDurationInDays`, `daysPerQuarter`, `yearsInCycle`, `getTimingInfo`: Cycle config helpers
- **Internals:** `_tryGetEquivalentEvtCount`, `_mapEquivalentEvtCountFromNftTypes`, `_getEvtApyBps`, `_getAccumulatedYield`, `_calcYieldSinceAmount`, `_getDepositApyBps`

FixedDepositsManager

- **Storage:** `fixedDeposits` address, three default EVT APY values (`defaultSingleEvtApyBps`, `defaultNineEvtApyBps`, `defaultFiftyFourEvtApyBps`)
- **Roles:** `DEFAULT_ADMIN_ROLE`, `APY_SYNC_ROLE`
- **Initialization:** Sets deposits contract, admin, and default APY values with validation
- **Core Functions:**
  - `addToWhitelist` / `removeFromWhitelist`: EIP-712 signature-gated whitelist management via `IFixedDepositsManagerCore`
  - `depositOnBehalf`: Signature-gated admin deposit with full COA verification
  - `resolveDepositApy`: EVT-based APY resolution called by `FixedDeposits` on every deposit
  - `syncUserApy`: `APY_SYNC_ROLE`-gated APY push for tier changes
- **Internals:** `_tryGetEquivalentEvtCount`, `_mapEquivalentEvtCountFromNftTypes`, `_getEvtApyBps`, `_validateApy`
